📝 React Final Project – Blog App

📘 Based on Lectures 1–4: JSX, Components, State, Props, Events, Routing, useEffect, API Integration, and Jotai

---

📌 Project Title:

Personal Blog App – A mini React application where users can browse blog posts fetched from a real API, view full post details with comments, create their own posts, and bookmark their favorites.

🧠 What You Will Practice:

Every concept from all 4 lectures is used in this project:

- **JSX & Components** — Building the UI with reusable pieces
- **useState** — Managing form data, bookmarks, and local state
- **Events** — Handling clicks and form submissions
- **Props** — Passing data between parent and child components
- **.map()** — Rendering lists of blog posts and comments
- **Conditional Rendering** — Showing placeholders, loading states, and error messages
- **React Router** — Navigating between pages
- **Dynamic Routing** — Viewing a specific post by its ID (`/blog/:id`)
- **useEffect + API** — Fetching real blog posts and comments from DummyJSON
- **Jotai** — Saving bookmarked posts globally so they appear on the Bookmarks page

---

🌐 The API — DummyJSON

You will use **DummyJSON** — the same API provider you already used in Lecture 3. No signup, no API key, completely free.

Here are the only 3 endpoints you need for this project:

Endpoint 1 — Get a list of blog posts (Home page)

```
https://dummyjson.com/posts?limit=10
```

- `limit=10` means fetch only 10 posts — no need to worry about pagination
- This returns a list of posts, each with an `id`, `title`, `body`, and `tags`

What the response looks like:


```
{
  "posts": [
    {
      "id": 1,
      "title": "His mother had always taught him",
      "body": "His mother had always taught him not to ever think of himself
as better than others...",
      "tags": ["history", "crime"],
      "reactions": { "likes": 192 },
      "userId": 121
    }
  ]
}
```


> 📌 To access the list use `data.posts` — the array is inside the `posts` key.

Endpoint 2 — Get a single post (Blog Details page)

```
https://dummyjson.com/posts/1
```

- Replace `1` with the actual post ID from the URL
- Returns one post with all its details

What the response looks like:


```
{
  "id": 1,
  "title": "His mother had always taught him",
  "body": "His mother had always taught him not to ever think...",
  "tags": ["history", "crime"],
  "reactions": { "likes": 192 },
  "userId": 121
}
```

Endpoint 3 — Get comments for a post (Blog Details page)

```
https://dummyjson.com/comments/post/1
```

- Replace `1` with the actual post ID
- Returns a list of comments for that post

What the response looks like:

```
{
  "comments": [
    {
      "id": 1,
      "body": "This is a great post!",
      "user": { "username": "sara123" }
    }
  ]
}
```

> 📌 To access the list use `data.comments` — the array is inside the `comments` key.

How to fetch in your component (same pattern from Lecture 3):

```
const [posts, setPosts] = useState([])
const [loading, setLoading] = useState(true)
const [error, setError] = useState(null)

useEffect(() => {
  fetch("https://dummyjson.com/posts?limit=10")
    .then((res) => {
      if (!res.ok) throw new Error("Failed to fetch")
      return res.json()
    })
    .then((data) => {
      setPosts(data.posts)   // ← the list is inside data.posts
      setLoading(false)
    })
    .catch((err) => {
      setError(err.message)
      setLoading(false)
```

```
    })
}, [])
```

📱 Pages & What Each Page Does:

1. Home Page (`/`)

- Fetches 10 blog posts from the API when the page loads
- Shows a loading message while fetching
- Shows an error message if the fetch fails
- Renders each post as a card showing the **title** and **tags**
- Clicking a post card navigates to its details page (`/blog/:id`)
- Includes a **"Create New Post"** button that goes to `/create`

---

2. Blog Details Page (`/blog/:id`)

- Gets the post `id` from the URL using `useParams()`
- Fetches the single post from `https://dummyjson.com/posts/:id`
- Fetches the comments from `https://dummyjson.com/comments/post/:id`
- Displays the post **title**, **body**, and **tags**
- Displays the list of **comments** below the post
- Includes a **Bookmark** button — adds/removes the post from global Jotai state
- Includes a **Back** button that goes back to Home

---

3. Create Post Page (`/create`)

- A simple form with two fields: **Title** and **Content**
- Basic validation — title cannot be empty
- On submit, the new post is added to a local list in state (no need to send to the API)
- After submitting, redirects back to Home using `useNavigate()`

---

4. Bookmarks Page (`/bookmarks`)

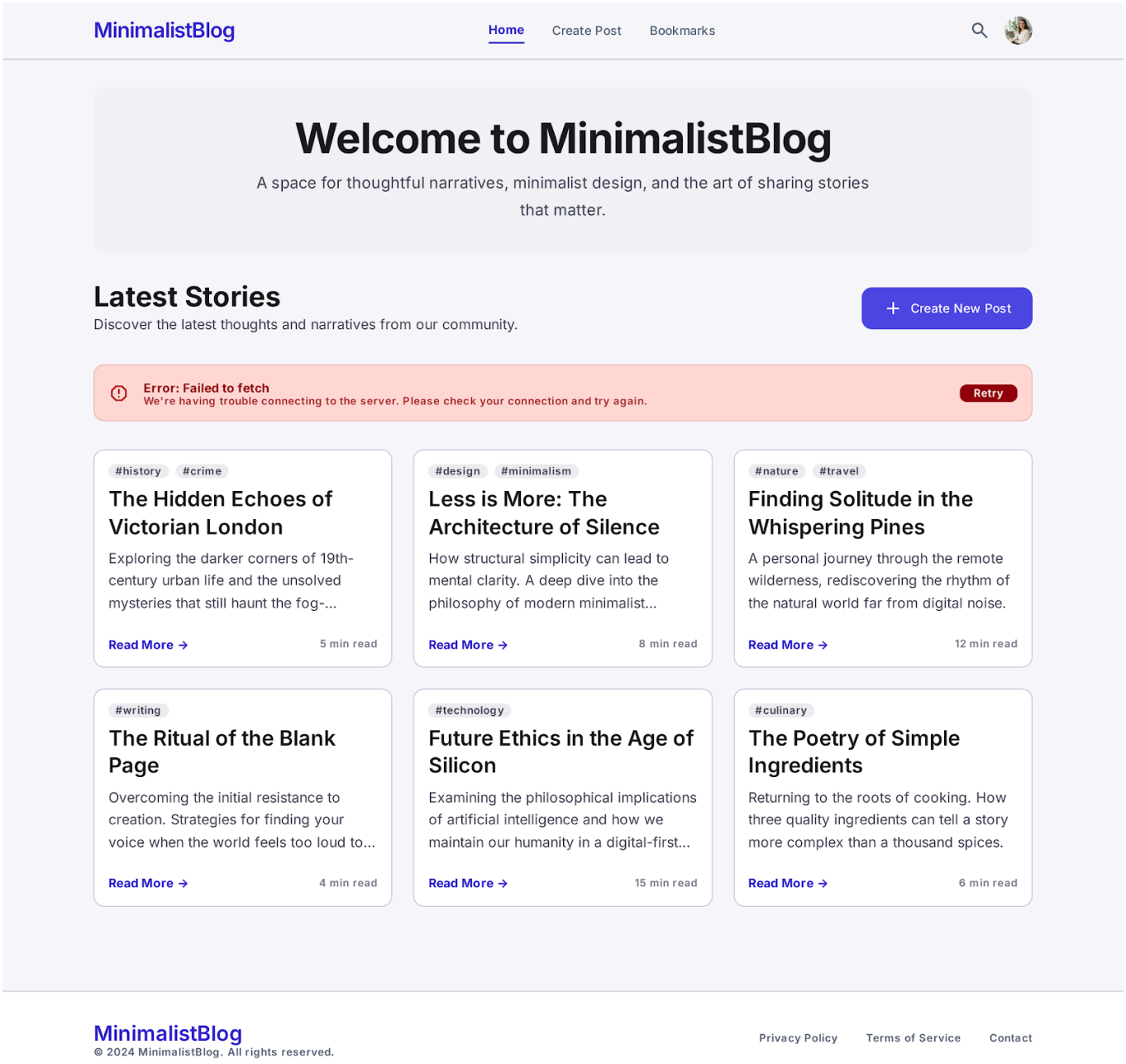
- Reads the bookmarked posts from the global Jotai atom
- Displays the list of bookmarked posts
- Shows "No bookmarks yet" if the list is empty
- Each bookmark links back to the full post details page

🎨 Sample Design

> The following design is a reference — you don't have to match it exactly.
> Use it as a guide for layout and structure.

> **NB: The design might include additional things that is not mentioned on 📱*Pages and What Each page Does*. Please ignore those added things on the design and only follow what is listed on the 📱*Pages and What Each page Does Section.***

1. Home Page



2. Blog Details Page

Back to Home

Design Productivity

Bookmark

The Silent Art of Minimalist Digital Design

In an era of constant digital noise, the most powerful designs are often those that say the least. Minimalism isn't just about what you remove; it's about what you choose to keep. By focusing on essential elements, we create space for the user's thoughts and intentions.

When we look at the landscape of modern applications, there's a trend toward sensory overload. Every pixel fights for attention. However, the truly premium experiences—the ones that stick—are those that respect the user's cognitive load.

> *"Minimalism is not the lack of something. It is simply the perfect amount of something."*

Implementing this philosophy requires discipline. It means choosing one typeface instead of three. It means using white space as a functional tool rather than just empty background. It means trusting that your content is strong enough to stand on its own without the crutch of decorative flourishes.

Comments 12

JD **@johndoe_design** 2h ago

This article perfectly encapsulates why I've been moving towards flatter UI patterns. The focus on 'digital quiet' is exactly what we need right now.

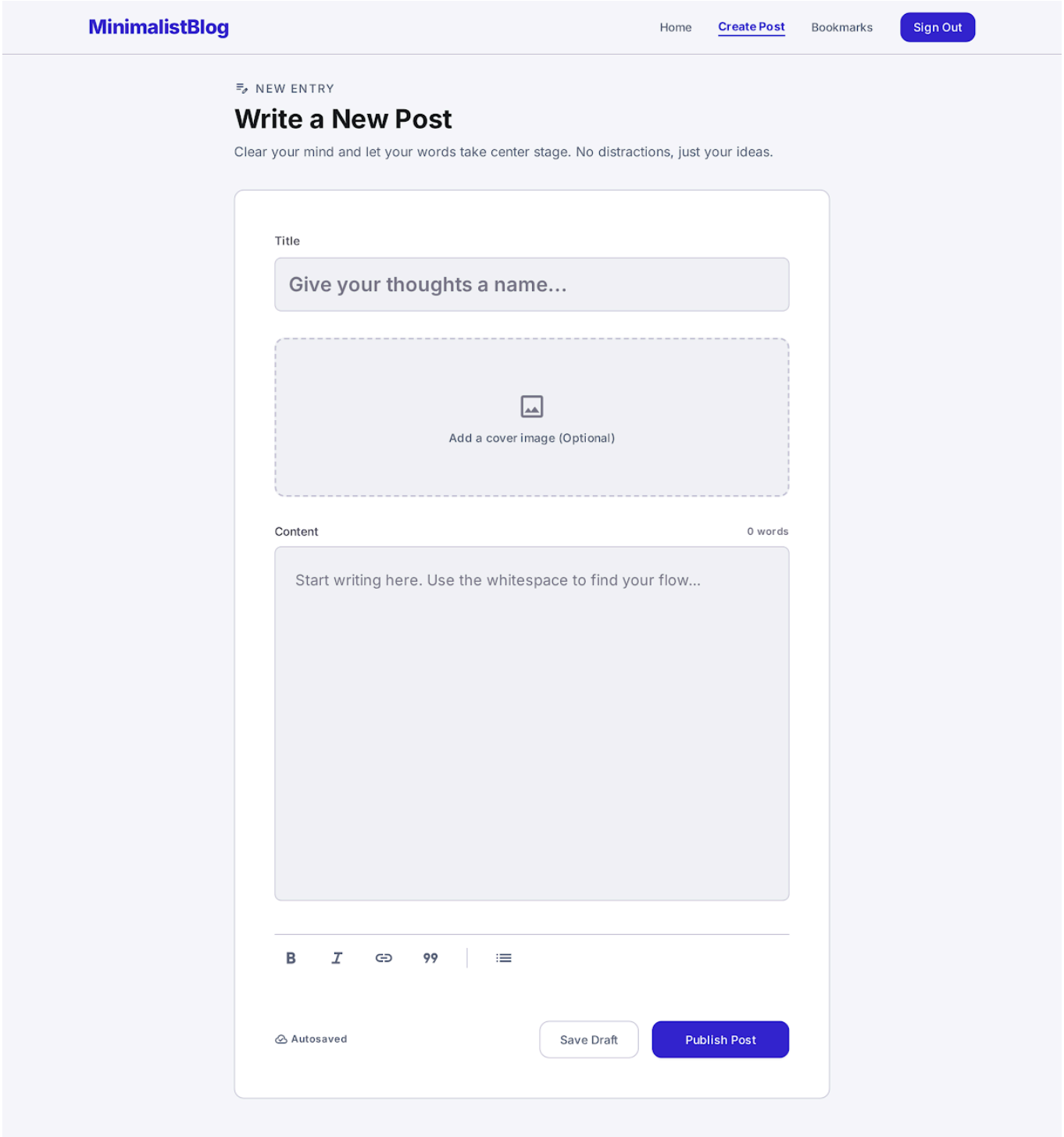
SM **@sarah_m** 5h ago

I love the point about white space being a functional tool. Often clients see it as wasted space, but this helps articulate why it's so vital for UX.

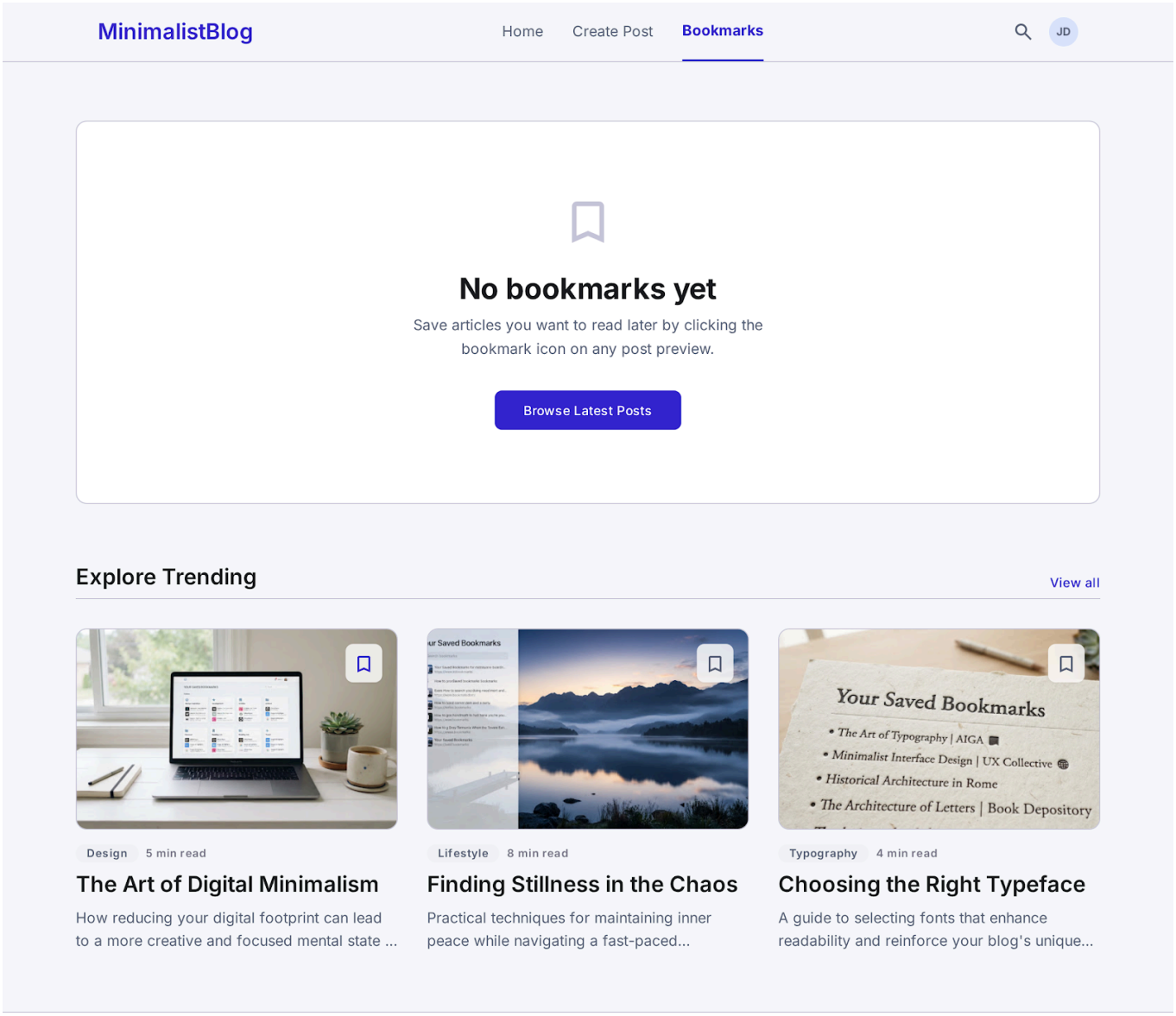
BK **@ben_kraft** 1d ago

The visual rhythm of this layout itself is a great example of what you're talking about. Very pleasant to read.

3. Create Post Page



4. Bookmarks Page



🧩 Components to Build:

`src/components/BlogCard.jsx`

- Receives a post as a prop
- Displays the title and tags
- The whole card is clickable and navigates to `/blog/:id`

`src/components/Navbar.jsx`

- Shows links to Home, Create Post, and Bookmarks
- Visible on every page

`src/components/BlogForm.jsx`

- A reusable form component used on the Create page
- Has inputs for Title and Content
- Receives an `onSubmit` function as a prop

⚛️ Global State with Jotai:

Create an atoms file to hold the bookmarks:

`src/atoms/bookmarkAtoms.jsx`

```
import { atom } from 'jotai'

// Stores the list of bookmarked posts
export const bookmarksAtom = atom([])
```

On the Blog Details page — add a bookmark:

```
const [bookmarks, setBookmarks] = useAtom(bookmarksAtom)

function handleBookmark() {
  const alreadyBookmarked = bookmarks.find((b) => b.id === post.id)
  if (!alreadyBookmarked) {
    setBookmarks((prev) => [ ...prev, post])
  }
}
```

On the Bookmarks page — read bookmarks:

```
const [bookmarks] = useAtom(bookmarksAtom)
```

📁 Suggested Folder Structure:

```
blog-app/
├── src/
│   ├── atoms/
│   │   └── bookmarkAtoms.jsx
│   ├── components/
```

```
│   │   ├── BlogCard.jsx
│   │   ├── BlogForm.jsx
│   │   └── Navbar.jsx
│   ├── pages/
│   │   ├── Home.jsx
│   │   ├── BlogDetails.jsx
│   │   ├── CreatePost.jsx
│   │   └── Bookmarks.jsx
│   ├── App.jsx
│   ├── main.jsx
│   └── index.css
```

✅ Marking Scheme (Total: /20)

Criteria	Description	Marks
JSX & Component Structure	Clean JSX, reusable components (BlogCard, Navbar, BlogForm)	2
Styling (Tailwind CSS)	Buttons, cards, layout and spacing styled properly	2
State & Props	`useState` for data, props passed correctly to children	2
API Integration	Posts and comments fetched from DummyJSON with loading and error handling	3
React Router & Dynamic Routing	All pages set up with correct routes, `/blog/:id` works correctly	2
Conditional Rendering	Loading state, error state, empty bookmarks message all handled	2
Create Post Form	Form works, validates empty title, adds post to local list, redirects after submit	2
Bookmark Functionality	Bookmarks managed with Jotai, visible on Bookmarks page	2
Deployment	App deployed on Vercel or Netlify, working live link submitted	2
Bonus	Clean UI, well-presented demo, or extra feature	+1
Late Submission	1 mark deducted per day without valid reason	-1
Total		**/20**

🚀 Bonus Ideas (Not required but encouraged):

- Add a **Remove Bookmark** button on the Bookmarks page
- Show the number of **likes** from the API on each blog card
- Add a **tag filter** on the Home page — click a tag to show only posts with that tag
- Show the **comment count** on each blog card

📤 Submission Instructions:

- Deploy your app on **Vercel** or **Netlify**
- Share your **live link** with your mentor
- Make sure all pages and routes work on the deployed version before submitting

Good luck — we can't wait to see what you build! 🚀